Tools Playbook: n8n, Make, Zapier, Replit, Claude and Codex

This playbook answers one question for each tool: **what do I do first, what do I ask for help, and where do I check the facts?** It is written so you can use it without any video. It uses fake data only.

> **Honesty note.** Tools change often. The steps below are stable habits. Feature names, plans and prices must be checked on the tool's own website the day you act. Community posts (for example Reddit) are useful for ideas, but check them against the official documentation before you rely on them.

1. One rule for every tool

1. **Write the goal in one sentence.**
2. **Use fake data and a test account.**
3. **Build one small step, test it, then add the next.**
4. **Ask for approval before anything is sent, deleted or changed for real.**

2. Choosing a tool for the job

Job	Start with	Why
Connect apps on a trigger, with branches and an approval step	n8n or Make	Visual flows, routing and error paths
Quick connections between popular apps, no code	Zapier	Large app library and simple setup
Build and host a small web app	Replit (or Lovable)	Code, preview and deploy in one place
Write, review or change code in a project	Claude Code or Codex	Agents that read files, run commands and propose changes
Think, write and plan	Claude or ChatGPT	Chat with projects, skills or instructions

3. How to ask for help (any tool)

Good questions get good answers. Give the AI these five things:

Part	Example
Tool and version	"n8n, self-hosted, latest I can see in the editor"
Goal in one line	"When a fake hire is posted, add one row to a test sheet."
What you did	"Webhook node, test URL, Sheets append node."
What you see	"The Sheets node shows an error: 'Column not found'."
What you want	"Tell me which field is wrong and the exact expression to fix it."

4. n8n: first steps and common errors

1. Create a workflow, add a **Webhook** node (POST, test URL), and click Listen for test event.
2. Send one fake body. Open the JSON view. Data sits under `body`.
3. Add a Sheets node. Map `{{ $json.body.hire_id }}` to a column.
4. Activate the workflow when the test passes, and switch the form to the production URL.

Error	Likely fix
"undefined" in a field	Check the path in the JSON view; add `body`
Nothing happens on a real event	Workflow not active, or the test URL is in use
Credential error	Reconnect the test account; check the permissions

5. Make: first steps and common errors

1. Create a scenario and add a trigger module (for example Google Forms: Watch Responses).
2. Run once after one fake submission. Check the bundle.
3. Add the action module and map the fields. Run once again.
4. Add a router with a fallback path, then turn the schedule on.

Error	Likely fix
Scenario is on but nothing runs	Check the trigger and schedule settings
Bundle disappears	Router with no matching filter and no fallback

Operations used up quickly	Lengthen the schedule; check the usage page

6. Zapier: first steps and common errors

1. Create a Zap with one trigger (the event that starts it) and one action.
2. Use a test account and test data. Run the test step.
3. Add filters or paths only after the first action works.
4. Turn the Zap on, then review its history after the first runs.

Error	Likely fix
Test step finds no data	Create a matching fake item in the source app and test again
Action fails on a field	Map a value to each required field
Duplicate items	Add a filter that checks an ID you store

7. Replit: first steps and common errors

1. Create a Repl from a template and press Run. Read the first red line in the console if it fails.
2. Store any API key in Secrets. Never paste it into code or chat.
3. Add one route, test it, then deploy and test the deployed address.

Error	Likely fix
Module not found	Add the package in the Packages panel and run again
Works in preview, fails deployed	Check the run command and secrets in deployment

Full steps are in the Replit guide in this series.

8. Claude: getting the most from it

- **Projects:** put standing instructions (tone, audience, rules) in the project once, instead of retyping them.
- **Skills:** save a repeated task as a skill with a header, rules, examples and a checklist. Use it again and again. See the Skills guide.
- **Context:** start a new chat when the topic changes. Check the context panel with `/context` in Claude Code.
- **Claude Code:** keep a `CLAUDE.md` file with the project's standing rules, so every session starts with them.

9. Codex: getting started safely

- Codex is an agent that can read a project, propose changes and run commands. Start in a copy of a practice project, never in your main code.
- Ask it to explain the plan before it edits files, then review each diff yourself.
- Keep secrets out of the project. Use environment settings for keys.
- Check OpenAI's current documentation for what Codex can access and how to limit it.

10. Where to learn more (check each site on the day)

Tool	Start here	Good for
n8n	docs.n8n.io	Nodes, webhooks, AI agents, hosting
Make	help.make.com	Scenarios, routers, data stores, AI
Zapier	zapier.com/help	Zaps, filters, app connections
Replit	docs.replit.com	Projects, secrets, deployment
Claude	docs.anthropic.com	Models, tools, skills, agents
Codex	OpenAI developer documentation	Agents that work on code

Community posts (for example Reddit discussions) can show real problems people hit. Use them to find questions to check, not as the final answer.

11. Ask-the-AI checklist

Before you ask	Done
I wrote the goal in one sentence	[ ]
I used fake data	[ ]
I included the exact error message	[ ]
I asked for one fix at a time	[ ]